Decision Trees and Ensemble Methods

Decision Trees

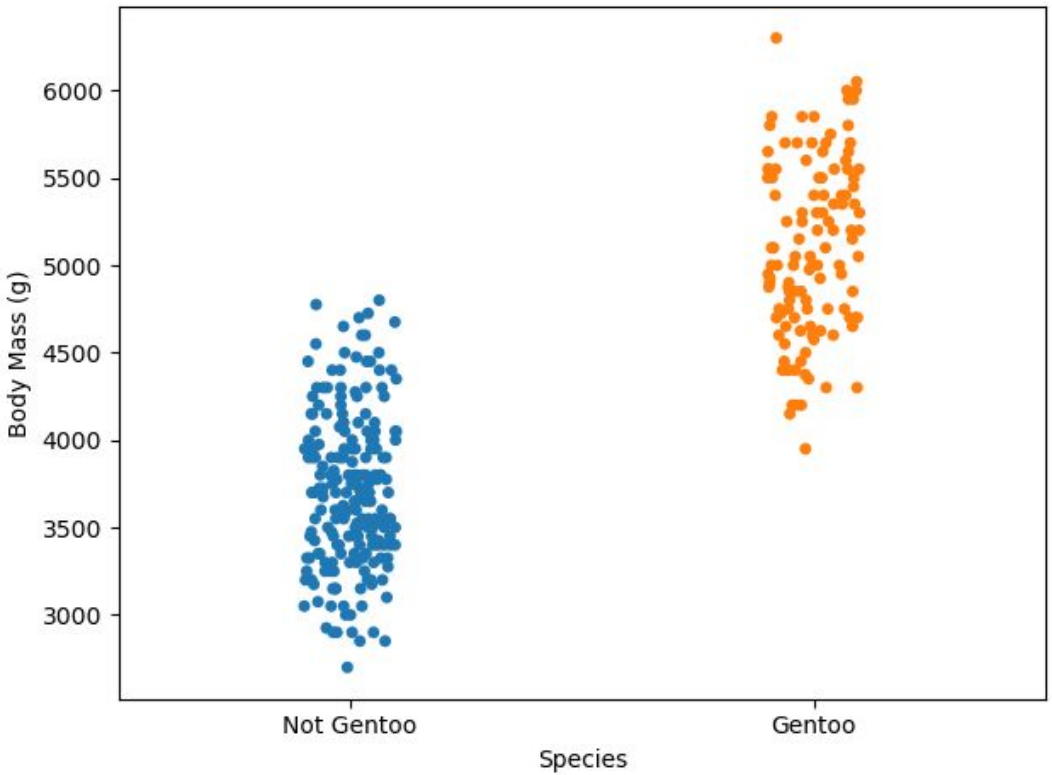
Big Idea: Ask a series of binary questions about one feature at a time to make a prediction.

Decision Trees - Example

Let’s say we are trying to distinguish gentoo penguins from the other species.

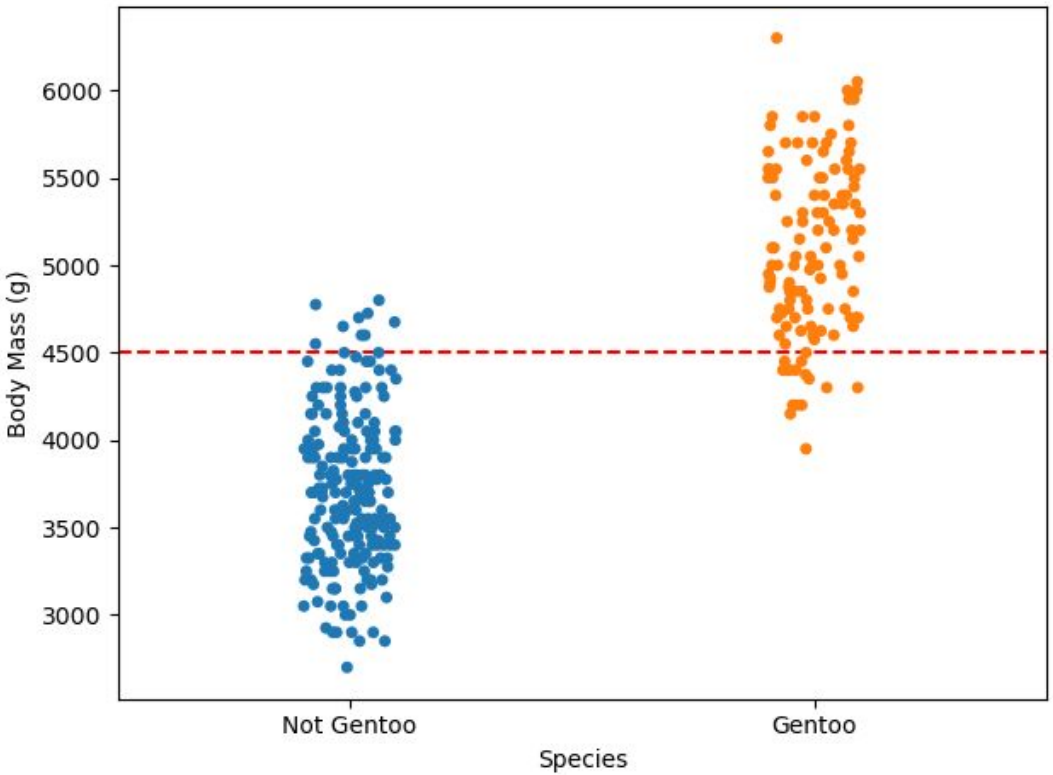
We have seen that gentoo penguins tend to have higher body mass than others, so let’s start by inspecting the body mass column.

Decision Trees - Example

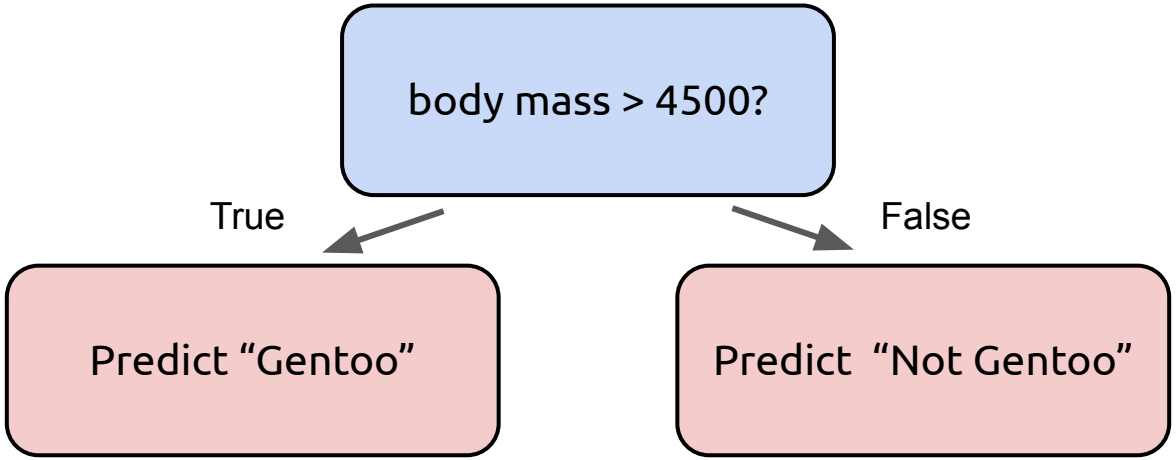


Perhaps we can find a cutoff for where to predict gentoo or not. Let’s use 4500 g.

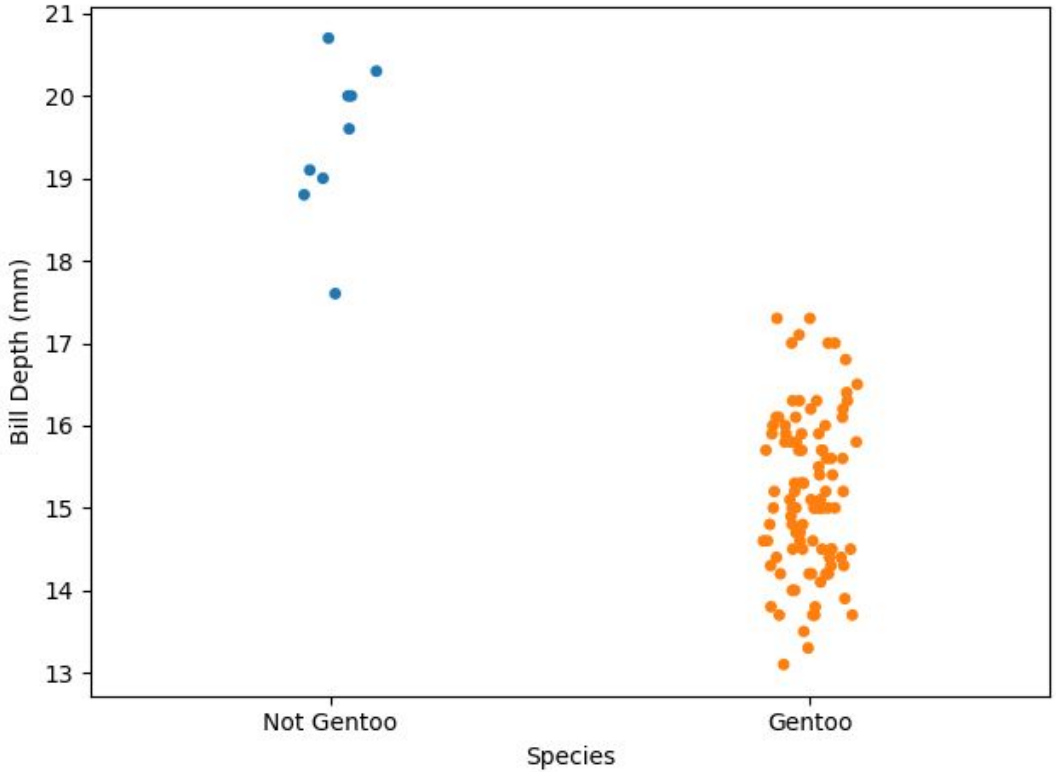
Decision Trees - Example



Perhaps we can find a cutoff for where to predict gentoo or not. Let’s use 4500 g.


body mass > 4500?
True
False
Predict “Gentoo”
Predict “Not Gentoo”

Decision Trees - Example

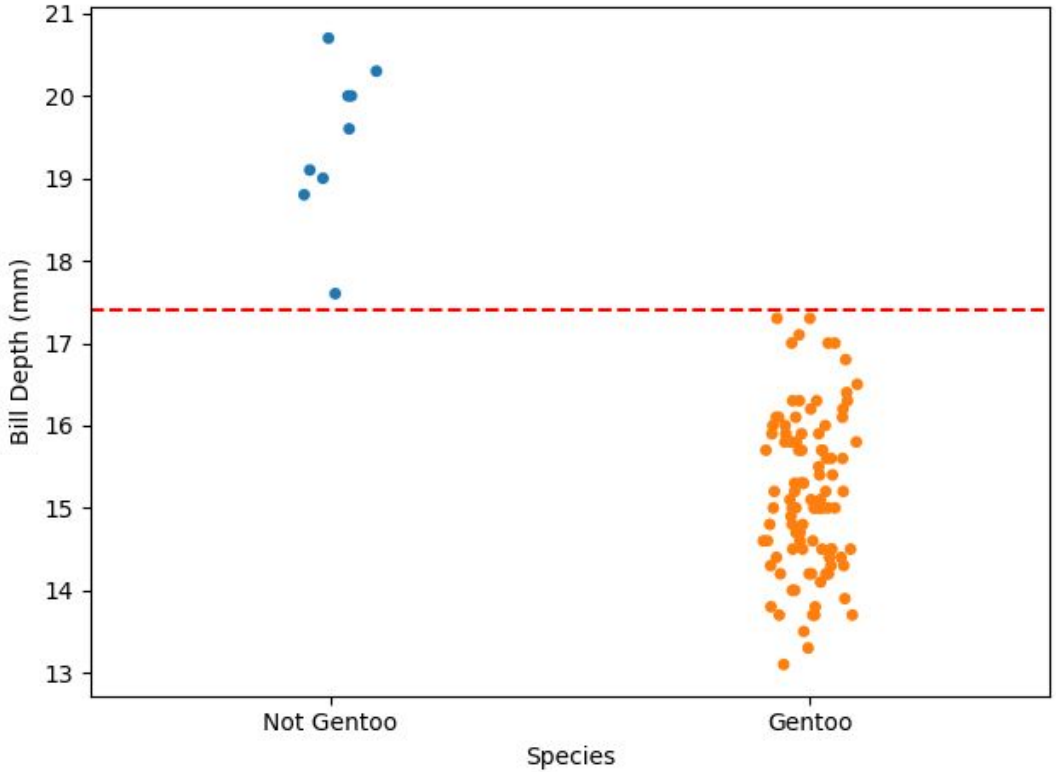


Let’s look at penguins whose body mass is larger than 4500 g.

If we focus on their bill depth, we see this.

Let’s use a cutoff of 17.4 mm.

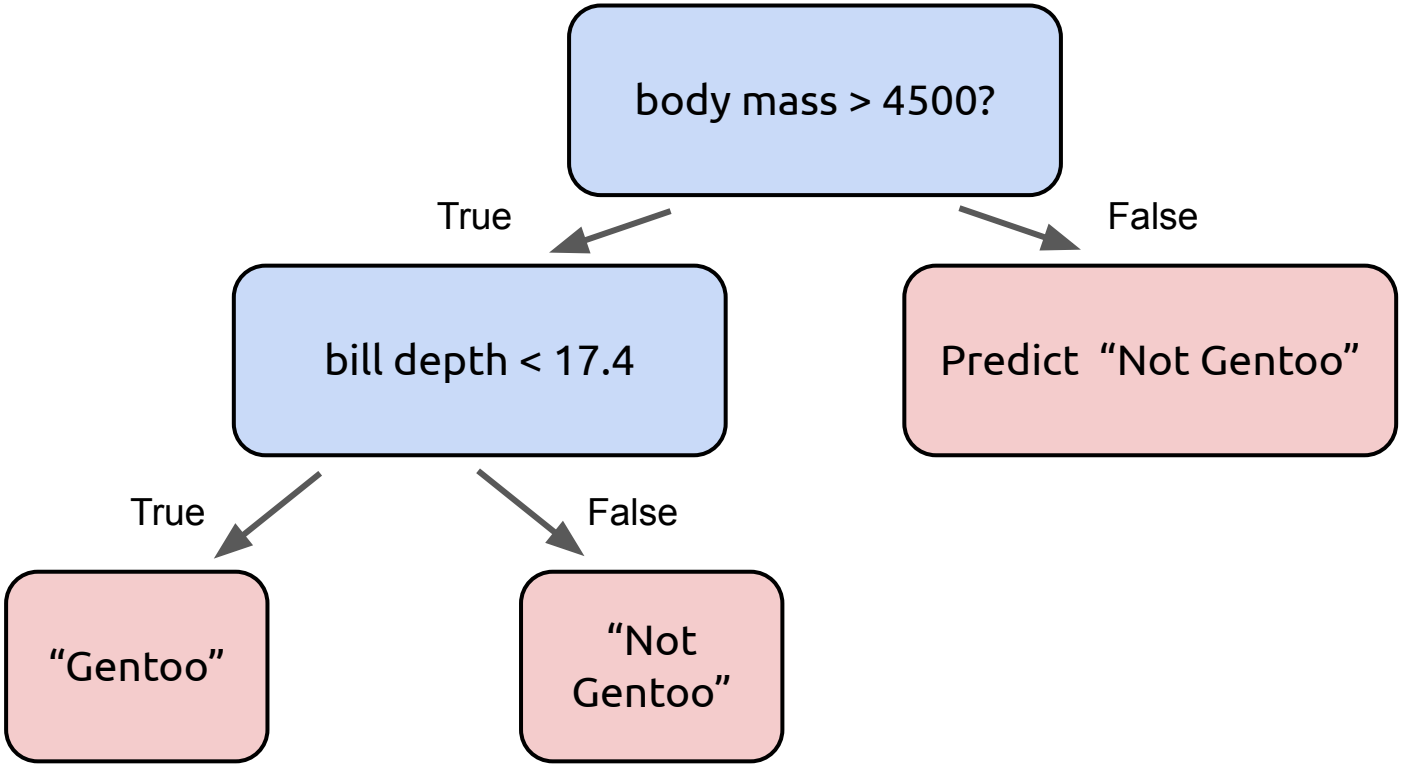
Decision Trees - Example



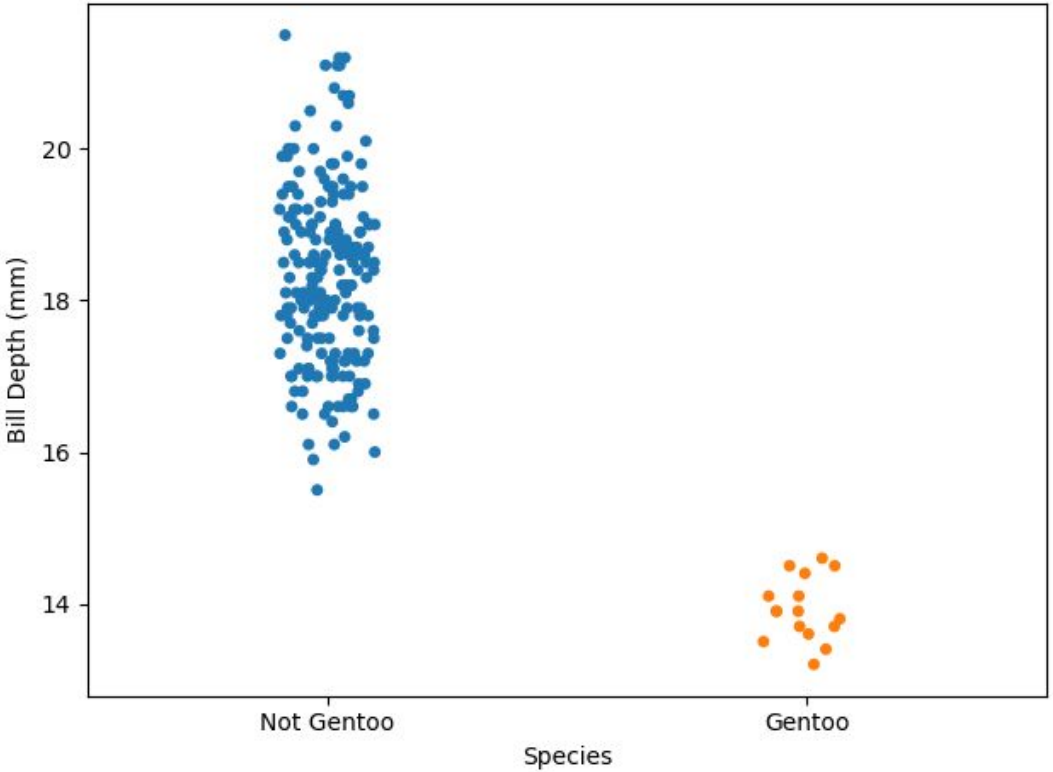
Let’s look at penguins whose body mass is larger than 4500 g.

If we focus on their bill depth, we see this.

Let’s use a cutoff of 17.4 mm.


body mass > 4500?
True
False
bill depth < 17.4
Predict "Not Gentoo"
True
False
"Gentoo"
"Not Gentoo"

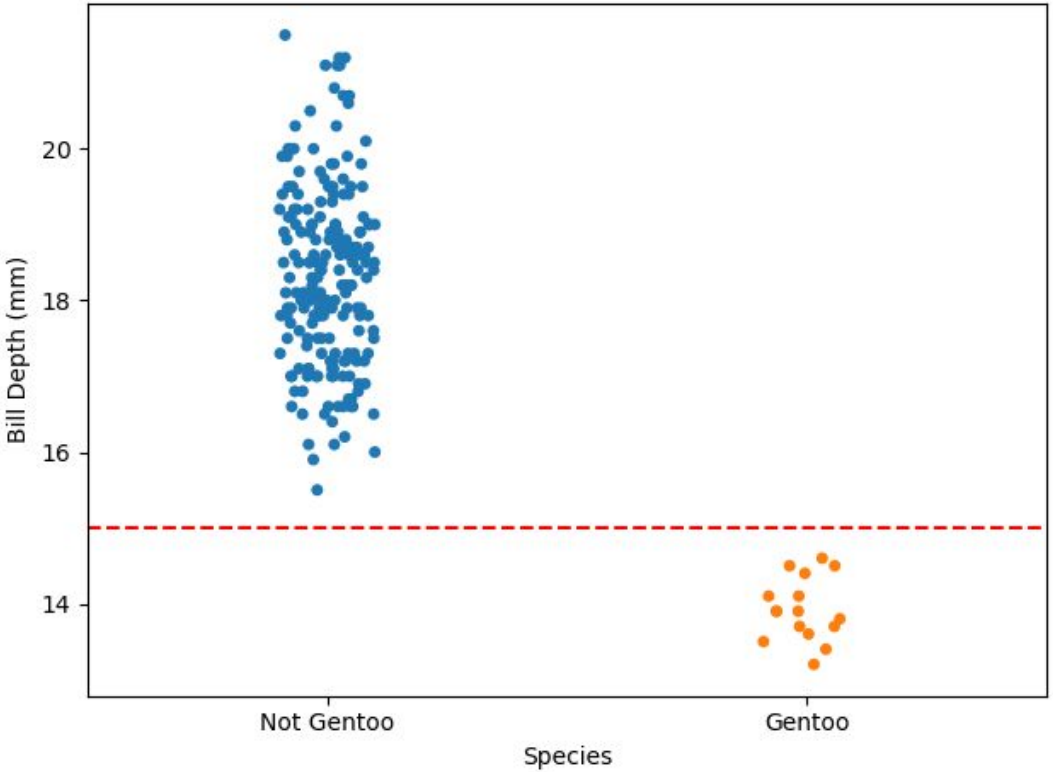
Decision Trees - Example



Now, those with body mass less than 4500.

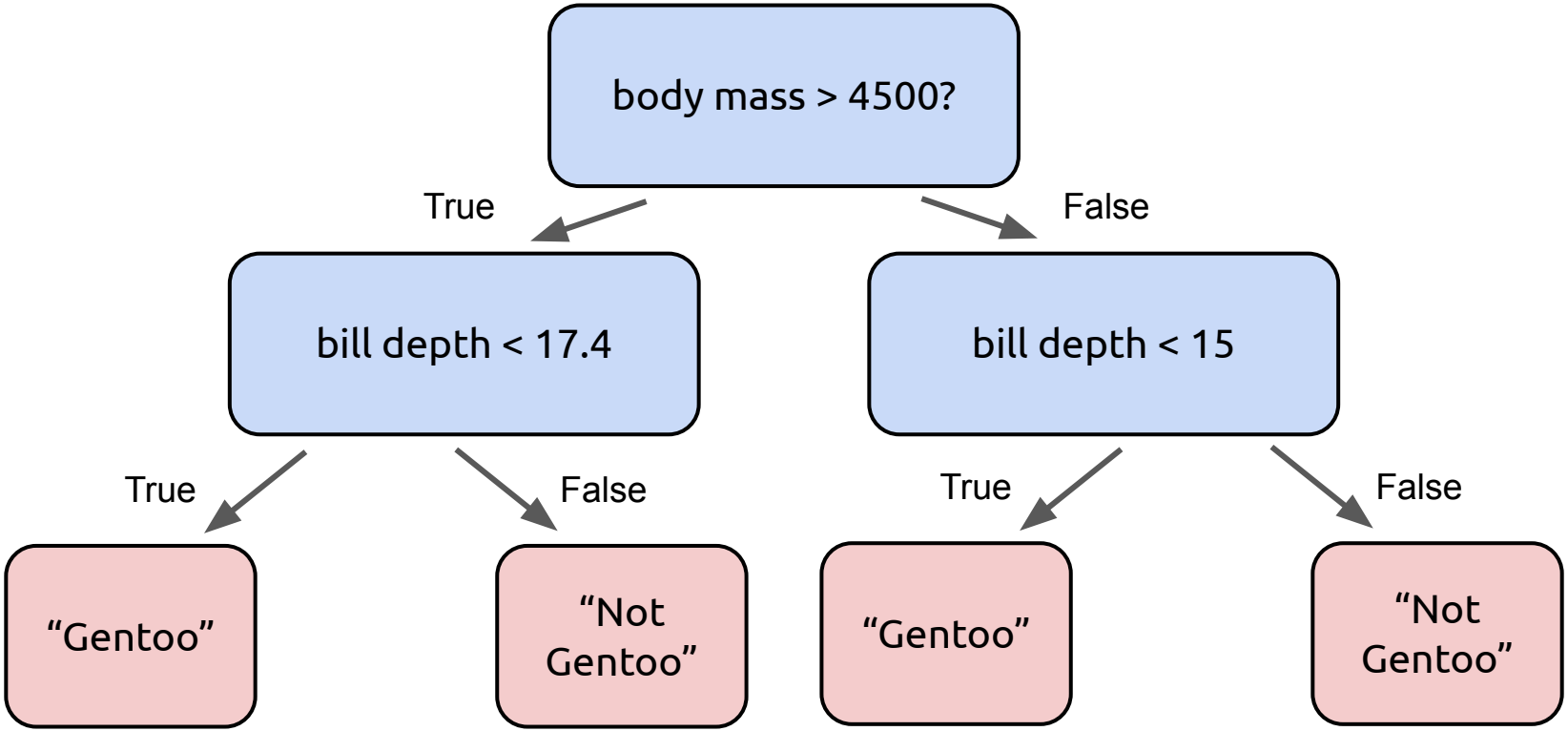
Here, we can use a cutoff of 15 mm.

Decision Trees - Example



Now, those with body mass less than 4500.

Here, we can use a cutoff of 15 mm.


body mass > 4500?
True
False
bill depth < 17.4
bill depth < 15
True
False
True
False
“Gentoo”
“Not Gentoo”
“Gentoo”
“Not Gentoo”

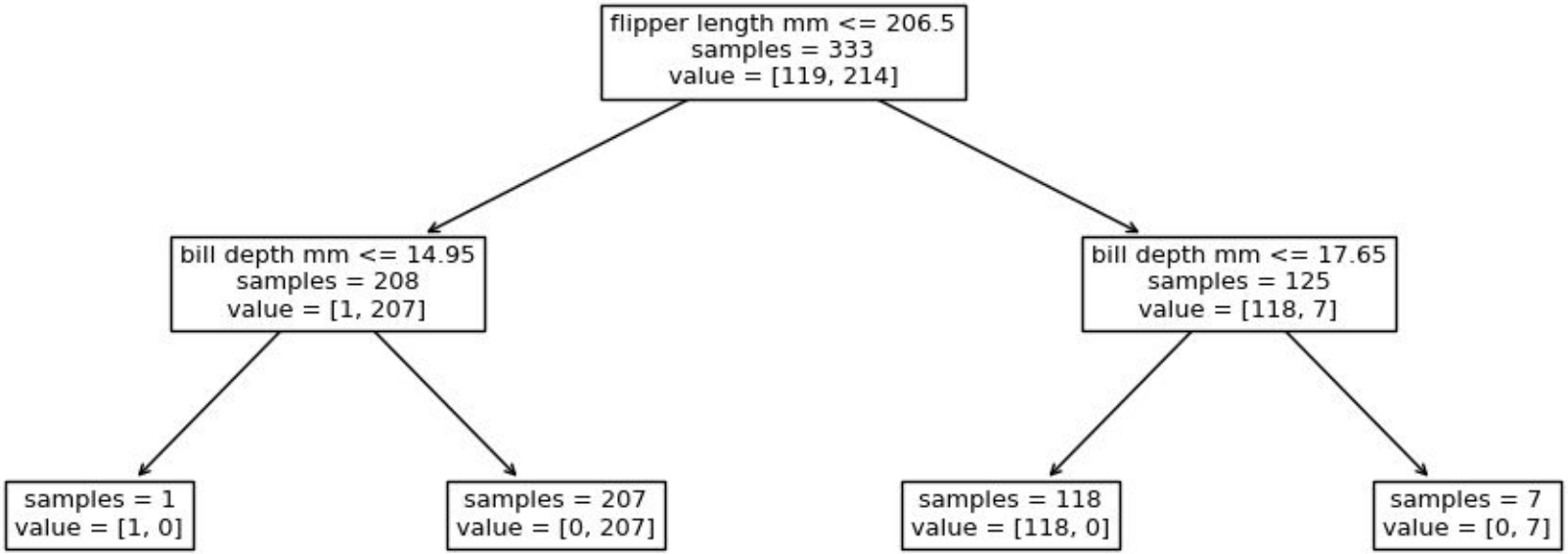
Decision Trees

Big Idea: Ask a series of binary questions about one feature at a time to make a prediction.

In actual practice, we wouldn’t do this manually, but instead allow an algorithm to determine which features and cutoffs to use.

Decision Trees

Here’s the tree that scikit-learn uses:



Decision Trees

Big Idea: Ask a series of binary questions about one feature at a time to make a prediction.

Pros:

- Simple
- Easy to understand how predictions are made
- Extremely Flexible

Cons:

- Very easy to overfit

Ensembling Methods

Ensemble Methods

Recall that in machine learning, an **ensemble** method is one which combines the results of many (usually simpler) predictors to generate one final prediction.

Today, we'll look at two types of ensemble methods that can be built off of decision trees:

- Random Forests
- Gradient Boosting

Random Forests

Big Idea: Train a large number of decision trees and average their predictions.

Each decision tree will only be fit on a random subset of the training data and a random subset of the available features.

Why? We want our predictions to be less correlated with each other.

This method is sometimes called bootstrap aggregation (**bagging**).

Gradient Boosting

We will look at another ensemble method, called **boosting**.

For bagging, our decision trees are built “in parallel” and predictions are aggregated at the end.

On the other hand, the decision trees for boosting are trained in sequence.

Big Idea: Each subsequent learner tries to correct the mistakes of the prior ones.

Boosting

Boosting Algorithm (for Regression):

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$
2. Generate predictions from the model and calculate the residuals:

$$r_i = y_i - f_0(\vec{x}_i)$$

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$
2. Generate predictions from the model and calculate the residuals:

$$r_i = y_i - f_0(\vec{x}_i)$$

3. Train a new model to predict the <u>residuals</u>**:** $f_1(x)$

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$
2. Generate predictions from the model and calculate the residuals:

$$r_i = y_i - f_0(\vec{x}_i)$$

3. Train a new model to predict the <u>residuals</u>**:** $f_1(x)$

$$f_0 + f_1 \approx f_0 + r = f_0 + y - f_0 = y$$

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$
2. Generate predictions from the model and calculate the residuals:

$$r_i = y_i - f_0(\vec{x}_i)$$

3. Train a new model to predict the <u>residuals</u>**:** $f_1(x)$
4. Update your model using (a shrunken version of) this new model:

$$f_0(x) \leftarrow f_0(x) + \lambda \cdot f_1(x)$$

Boosting

Boosting Algorithm (for Regression):

1. Fit an initial model to the data: $f_0(\vec{x})$
2. Generate predictions from the model and calculate the residuals:

$$r_i = y_i - f_0(\vec{x}_i)$$

3. Train a new model to predict the <u>residuals</u>**:** $f_1(x)$
4. Update your model using (a shrunken version of) this new model:

$$f_0(x) \leftarrow f_0(x) + \lambda \cdot f_1(x)$$

5. GOTO 2

Boosting Example

Let's say we are building a model to predict home prices. We have a very small training set of only 4 values, but decide to proceed anyway. We train our initial model $f_0(x)$ (say a simple decision tree model)

True Value y_i	Predicted Value $f_0(x_i)$	Residual
$190,000	$250,000	
$215,000	$205,000	
$280,000	$305,000	
$320,000	$275,000	

Boosting Example

Now, we can see how far off we were. That is, we can find the residuals.

True Value y_i	Predicted Value $f_0(x_i)$	Residual $r_i = y_i - f_0(x_i)$
\$190,000	\$250,000	-\$60,000
\$215,000	\$205,000	\$10,000
\$280,000	\$305,000	-\$25,000
\$320,000	\$275,000	\$45,000

Boosting Example

Now, we build a new model, $f_1(x)$, to try and predict the residuals:

Residual r_i	Predicted Residual $f_1(x_i)$
-$60,000	-$48,000
$10,000	$9,000
-$25,000	-$28,000
$45,000	$49,000

Boosting Example

Combine f_0 and f_1 to get our new predicted values:

True Value y_i	Initial Predicted Value $f_0(x_i)$	Predicted Residual $f_1(x_i)$	Final Predicted Value $f_0(x_i) := f_0(x_i) + f_1(x_i)$
$190,000	$250,000	-$48,000	
$215,000	$205,000	$9,000	
$280,000	$305,000	-$28,000	
$320,000	$275,000	$49,000	

Boosting Example

Combine f_0 and f_1 to get our new predicted values:

True Value y_i	Initial Predicted Value $f_0(x_i)$	Predicted Residual $f_1(x_i)$	Final Predicted Value $f_0(x_i) := f_0(x_i) + f_1(x_i)$
$190,000	$250,000	-$48,000	$202,000
$215,000	$205,000	$9,000	$214,000
$280,000	$305,000	-$28,000	$277,000
$320,000	$275,000	$49,000	$324,000

Boosting Example

Now, we can use our new $f_0(x)$ to compute the new residuals and repeat the process.

True Value y_i	Predicted Value $f_0(x_i)$	Residual $r_i = y_i - f_0(x_i)$
$190,000	$202,000	
$215,000	$214,000	
$280,000	$277,000	
$320,000	$324,000	

Boosting Example

Now, we can use our new $f_0(x)$ to compute the new residuals and repeat the process.

True Value y_i	Predicted Value $f_0(x_i)$	Residual $r_i = y_i - f_0(x_i)$
$190,000	$202,000	-$12,000
$215,000	$214,000	$1,000
$280,000	$277,000	$3,000
$320,000	$324,000	-$4,000

Boosting

Why do we need λ?

It acts as a form of regularization, helping prevent overfitting. Instead of taking the whole new prediction, we take only a fraction of it.

Small values of lambda (0.01 or below) can help avoid overfitting, at the expense of increased training time.

$$f_0(x) \leftarrow f_0(x) + \boxed{\lambda} \cdot f_1(x)$$

Difference Between Boosting and Random Forests

Random Forest:

- Trees are trained independently.
- Final predictions are obtained by averaging the predictions from all trees.

Boosting:

- Trees are trained in sequence.
- Final predictions are obtained by taking a weighted sum of individual trees.

Other Implementations of Gradient Boosting

XGBoost: released in 2014 and gained popularity after winning the Higgs Machine Learning Challenge, where the goal was to classify signals generated by particle colliders.

In addition to the basic gradient boosting algorithm, it adds some additional tweaks, leading to stronger overall performance:

- Regularization (think LASSO or Ridge Regression)
- Better search algorithm to train component decision trees
- Structure allowing for parallel computing for faster training

https://xgboost.readthedocs.io/en/stable/python/python_intro.html

Other Implementations of Gradient Boosting

LightGBM: Developed by Microsoft. Focused on performance and scalability.

One of the more popular models to use by Kaggle competitors.

https://lightgbm.readthedocs.io/en/latest/Python-Intro.html